



Rebasing <cmath> on C23

Document number:

[P3935R1](#)

Date:

2026-05-11

Audience:

SG22

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

GitHub Issue:

wg21.link/P3935/github

Source:

github.com/eisenwave/cpp-proposals/blob/master/src/cmath-c23.cow



ABSTRACT: Most of C++26 is based on C23, but the header <cmath> is still based on C17. There are many useful C23 <math.h> features that should be provided in C++.

§ Contents

- 1 **Revision history**
 - 1.1 Changes since R0
- 2 **Introduction**
- 3 **Design**
 - 3.1 New functions
 - 3.1.1 No SIMD support
 - 3.1.2 Narrow rounding functions
 - 3.1.3 Canonical floating-point representations and canonicalization
 - 3.2 New macros
 - 3.3 New declarations for `floatN_t`
- 4 **Implementation experience**
- 5 **Wording**
 - 5.1 [basic.fundamental]
 - 5.2 [version.syn]

5.3	[cmath.syn]
5.4	[c.math.narrow]
5.5	[sf.cmath]
6	Acknowledgements
7	References



§ 1. Revision history

- Removed `noexcept` from `std::fadd` and other proposed templates
- Added §3.1.1. No SIMD support

§ 1.1. Changes since R0

§ 2. Introduction

[P3348R4] rebased the C++26 standard on C23; it previously referred to C17. However, this process was deliberately left incomplete: some of the new C23 `<math.h>` features are only used by decimal floating-point types, or they require core language changes, etc.

The goal of this proposal is to pull in all the new C23 `<math.h>` features which are useful not only to decimal floating-point numbers.

Introducing decimal floating-point numbers into C++ is out-of scope. That is, adding C's `_Decimal128` type as a `std::decimal128_t` alias is not proposed. However, this proposal lays a bit of groundwork to make that happen in the future, such as introducing the concept of canonical representations into the core language.

`<simd>` is also out-of-scope, but expanding `<simd>` support for these new functions should be done in a future paper.

§ 3. Design

§ 3.1. New functions

The functions in the table below are proposed for inclusion in C++29. The placeholder *F* is a shorthand for the *floating-point-type* placeholder in [cmath.syn]. In addition to the functions using an *F* placeholder, single-type functions such as `float acospi(float)` are proposed, but are not listed in the table for the sake of brevity; refer to §5. Wording for the complete list.

Function	ISO/IEC 60559 op.	Returns
----------	-------------------	---------

<code>bool iscanonical(<i>F</i> x);</code>	isCanonical	true if and only if <i>x</i> is canonical 
<code>bool issignaling(<i>F</i> x);</code>	isSignaling	true if and only if <i>x</i> is a signaling NaN
<code>bool issubnormal(<i>F</i> x);</code>	isSubnormal	true if and only if <i>x</i> is subnormal
<code>bool iszero(<i>F</i> x);</code>	isZero	true if and only if <i>x</i> is zero
<code><i>F</i> acospi(<i>F</i> x);</code>	acosPi	$\arccos(x) \div \pi$ in the interval $[0, 1]$
<code><i>F</i> asinpi(<i>F</i> x);</code>	asinPi	$\arcsin(x) \div \pi$ in the interval $\left[-\frac{1}{2}, +\frac{1}{2}\right]$
<code><i>F</i> atanpi(<i>F</i> x);</code>	atanPi	$\arctan(x) \div \pi$ in the interval $\left[-\frac{1}{2}, +\frac{1}{2}\right]$
<code><i>F</i> atan2pi(<i>F</i> y, <i>F</i> x);</code>	atan2Pi	$\text{atan2}(x, y) \div \pi$ in the interval $[-1, +1]$
<code><i>F</i> cospi(<i>F</i> x);</code>	cosPi	$\cos(x \times \pi)$
<code><i>F</i> sinpi(<i>F</i> x);</code>	sinPi	$\sin(x \times \pi)$
<code><i>F</i> tanpi(<i>F</i> x);</code>	tanPi	$\tan(x \times \pi)$
<code><i>F</i> exp10(<i>F</i> x);</code>	exp10	10^x
<code><i>F</i> exp10m1(<i>F</i> x);</code>	exp10m1	$10^x - 1$
<code><i>F</i> exp2m1(<i>F</i> x);</code>	exp2m1	$2^x - 1$
<code><i>F</i> log10p1(<i>F</i> x);</code>	log10p1	$\log_{10}(1 + x)$
<code><i>F</i> logp1(<i>F</i> x);</code>	logp1	$\log(1 + x)$
<code><i>F</i> log2p1(<i>F</i> x);</code>	log2p1	$\log_2(1 + x)$
<code><i>F</i> compoundn(<i>F</i> x, long long n);</code>	compound	$(1 + x)^n$
<code><i>F</i> pown(<i>F</i> x, long long n);</code>	pown	x^n
<code><i>F</i> powr(<i>F</i> y, <i>F</i> x);</code>	powr	$e^{y \log_e x}$
<code><i>F</i> rootn(<i>F</i> x, long long n);</code>	rootn	$x^{\frac{1}{n}}$
<code><i>F</i> rsqrt(<i>F</i> x);</code>	rSqrt	$\frac{1}{\sqrt{x}}$
<code><i>F</i> roundeven(<i>F</i> x);</code>	roundToIntegralTiesToEven	<i>x</i> rounded to the nearest integer, rounding halfway cases to even
<code><i>F</i> fromfp(<i>F</i> x, int rnd, unsigned width);</code>	convertToInteger...	<i>x</i> rounded to a signed width-bit integer, or NaN if <i>x</i> is not in range; rnd is the rounding mode
<code><i>F</i> ufromfp(<i>F</i> x, int rnd, unsigned width);</code>	convertToInteger...	<i>x</i> rounded to an unsigned width-bit integer,

		or NaN if x is not in range; rnd is the rounding mode
<i>F</i> fromfp(<i>F</i> x, int rnd, unsigned width);	convertToIntegerExact...	fromfp(x, rnd, width); may also raise FE_INEXACT
<i>F</i> ufromfp(<i>F</i> x, int rnd, unsigned width);	convertToIntegerExact...	ufromfp(x, rnd, width); may also raise FE_INEXACT
int canonicalize(<i>F*</i> cx, const <i>F*</i> x);	convertFormat (same format)	zero if canonicalization succeeded, and nonzero value otherwise
<i>F</i> fmaximum_mag(<i>F</i> x, <i>F</i> y);	maximumMagnitude	the value with greater magnitude, or fmaximum(x, y)
<i>F</i> fminimum_mag(<i>F</i> x, <i>F</i> y);	minimumMagnitude	the value with greater magnitude, or fminimum(x, y)
<i>F</i> fmaximum_mag_num(<i>F</i> x, <i>F</i> y);	maximumMagnitudeNumber	the value with greater magnitude, or fmaximum_num(x, y)
<i>F</i> fminimum_mag_num(<i>F</i> x, <i>F</i> y);	minimumMagnitudeNumber	the value with greater magnitude, or fminimum_num(x, y)
float fadd(<i>F</i> x, <i>F</i> y); double dadd(<i>F</i> x, <i>F</i> y);	addition	$x + y$
float fsub(<i>F</i> x, <i>F</i> y); double dsub(<i>F</i> x, <i>F</i> y);	subtraction	$x - y$
float fmul(<i>F</i> x, <i>F</i> y); double dmul(<i>F</i> x, <i>F</i> y);	multiplication	$x \times y$
float fdiv(<i>F</i> x, <i>F</i> y); double ddiv(<i>F</i> x, <i>F</i> y);	division	$x \div y$
float ffma(<i>F</i> x, <i>F</i> y, <i>F</i> z); double dfma(<i>F</i> x, <i>F</i> y, <i>F</i> z);	fusedMultiplyAdd	$(x \times y) + z$
float fsqrt(<i>F</i> x); double dsqrt(<i>F</i> x);	squareRoot	$\sqrt{x}$
bool iseqsig(<i>F</i> x, <i>F</i> y)	compareSignalingEqual	true if x and y are equal and false otherwise
bool totalorder(const <i>F*</i> x, const <i>F*</i> y);	totalOrder	true if totalOrder(*x, *y) is true, as per ISO/IEC 60559
bool totalordermag(const <i>F*</i> x, const <i>F*</i> y);	totalOrderMag	true if totalOrderMag(*x, *y) is true, as per ISO/IEC 60559

<code>F getpayload(const F* x);</code>	<code>getPayload</code>	the NaN payload in <code>*x</code> , or -1 if the <code>*x</code> is not a NaN
<code>int setpayload(F* res, F pl);</code>	<code>setPayload</code>	zero if the NaN payload in <code>*res</code> could be set, and nonzero value otherwise
<code>int setpayloadsig(F* res, F pl);</code>	<code>setPayloadSig</code>	same as <code>setpayload</code> , but creates sNaN instead of qNaN

As can be seen, all of these implement an ISO/IEC 60559 operation, and even if they didn't, they would be useful solely for the purpose of C compatibility.

All functions below `iseqsig` are specific to ISO/IEC 60559, and are only provided by C23 for types that adhere to ISO/IEC 60559. The same restriction should be inherited by C++; otherwise e.g. `totalorder` would require us to define a total order for every possible format of floating-point type, which seems unreasonable.

All new functions should be `constexpr` because none of them have a strong dependence on the current rounding mode or floating-point environment.

§ 3.1.1. No SIMD support

At this time, no SIMD overloads are proposed because

- it would significantly broaden the scope of this proposal beyond just establishing C23 compatibility,
- the function signatures are not obvious in many cases (would `fromfp` take a `basic_vec` of rounding modes?), and
- there is no particular urgency to do it all in one paper; it does not result in a broken specification, just in some missing nice-to-have features for `<simd>`.

§ 3.1.2. Narrow rounding functions

The narrow rounding functions (such as `fadd`) are unusually difficult to expose in C++ in a generic way. Even the `<tgmath.h>` versions require the user to specify the return type as a prefix (`f`, `d`, `f32`, ...) as part of the function name. This is extremely hostile to generic code, and also means there is no support for extended floating-types in C++ because we don't consider `std::float32_t` to be C23's `_Float32` and to have `sqrtf32` and other functions that accept it.

To solve this, we can do something novel: add function templates for these rounding functions:

```
template<class T, class F1, class F2>
constexpr T fadd(F1 x, F2 y);
template<class T, class F1, class F2>
constexpr T fsub(F1 x, F2 y);
template<class T, class F1, class F2>
constexpr T fmul(F1 x, F2 y);
template<class T, class F1, class F2>
```

```
constexpr T fdiv(F1 x, F2 y);
template<class T, class F1, class F2, class F3>
constexpr T fma(F1 x, F2 y, F3 z);
template<class T, class F1>
constexpr T fsqrt(F1 x);
```



While the prefix “f” is a bit inappropriate (it is supposed to indicate a `float` return type), there is no obvious alternative prefix, “f” definitely won’t clash with future `<math.h>` functions, and having no prefix at all would create undesirable overload sets with other functions.

The design goal for these functions is for the user to be able to write:

```
#ifdef __cplusplus
#define f32add(...) ::std::fadd<std::float32_t>(__VA_ARGS__)
// ...
#else
#include <tgmath.h>
#endif
```

There is also a non-template `std::fadd` function which is meant to emulate the behavior of the `<tgmath.h>` macro by selecting between the `long double→float` and `double→float` overloads.



NOTE: These templates are not `noexcept` because they dispatch to underlying C functions that are not `noexcept`. We also traditionally don’t make new C++ overloads of C functions `noexcept`, such as in the case of the three-parameter `std::lerp` overload. Despite that, they are not intended to throw.

§ 3.1.3. Canonical floating-point representations and canonicalization

C23 introduced the concept of “canonical representations” identified by `iscanonical`, and a function `canonicalize` to convert to canonical representations of the same value.



EXAMPLE: Examples of canonical/non-canonical representations include:

- Representations for decimal floating-point types are specified to be canonical or non-canonical by ISO/IEC 60559.
- Signaling NaN may be non-canonical, whereas quiet NaN may be canonical.
- An implementation that flushes subnormals to zero may consider all subnormal normals to be non-canonical representations of zero.
- Negative zero may be considered a non-canonical representation of positive zero, which is canonical.

Unless working with decimal floating-point types, the classification of representations as “canonical” is largely based on the subjective preferences of the implementation; there are no hard rules other than a canonical representation being unique.

§ 3.2. New macros



Besides the new functions, there are also macros which are pulled in from C23.



NOTE: The `int rnd` parameter in functions such as `fromfp` corresponds to one of `FP_INT_*` rounding direction macros.

The block below lists all new macros (not yet including the `floatN_t` placeholder versions):

```
// rounding direction macros
FP_INT_UPWARD
FP_INT_DOWNWARD
FP_INT_TOWARDZERO
FP_INT_TONEARESTFROMZERO
FP_INT_TONEAREST

// indicates that the respective function is "fast"
FP_FAST_FADD
FP_FAST_FADDL
FP_FAST_DADDL
FP_FAST_FSUB
FP_FAST_FSUBL
FP_FAST_DSUBL
FP_FAST_FMUL
FP_FAST_FMULL
FP_FAST_DMULL
FP_FAST_FDIV
FP_FAST_FDIVL
FP_FAST_DDIVL
FP_FAST_FSQRT
FP_FAST_FSQRTL
FP_FAST_DSQRTL
FP_FAST_FFMA
FP_FAST_FFMAL
FP_FAST_DFMAL
```

§ 3.3. New declarations for `floatN_t`

C23 also provides a suffixed version of all the common mathematical functions with `fN` suffix and with `floatN_t` parameters. While these are not particularly useful considering that the user can (and probably should) call the overloaded functions, the current design direction is for C++ to inherit suffixed functions; we already do so for `sqrtf` and `sqrtl`. It would make porting C code to C++ and vice versa needlessly difficult if the suffixed functions only existed in one standard, for seemingly no technical reason.



NOTE: In the case of the narrow rounding functions, C23 also has

- prefixed generic functions such as `f32add`, and
- prefixed concrete functions such as `f32addf64`,

as explained above.



The pattern of always providing the suffixed versions is only broken by `fminimum`, `fminimum_num`, `fmaximum`, and `fmaximum_num`. These functions were added to C++26 by [P3008R6], mostly to specify the behavior of atomic floating-point minimum/maximum in terms of the C23 functions. There is no technical reason why the suffixed versions shouldn't also be provided, and the paper does not discuss this option.

§ 4. Implementation experience

All non-template additions are taken from C23, and most have been implemented in glibc.

The new narrow rounding function templates simply dispatch to some non-template overload based on the template argument `T`.

§ 5. Wording

The changes are relative to [N5014].

§ [basic.fundamental]

Immediately prior to [basic.fundamental] paragraph 13, insert a new paragraph:



An implementation may prefer particular representations of values that have multiple representations in a floating-point type. The preferred representations of a floating-point type, including unique representations of values in the type, are called *canonical*. A floating-point type may also contain *non-canonical representations*, for example, redundant representations of some or all its values, or representations that are extraneous to the floating-point model. Typically, floating-point operations deliver results with canonical representations.

[Note: The functions `std::iscanonical` and `std::canonicalize` distinguish canonical (preferred) representations, but this distinction alone does not imply that canonical and non-canonical representations are of different values. — *end note*]



DRAFTING NOTE: The C23 wording is extremely vague, and we can't be precise either if we don't want to risk having diverging behavior for `iscanonical`.

`iscanonical` and `canonicalize` are primarily motivated by ISO/IEC 60559 decimal floating-point representations anyway, where it is precisely specified which representations are canonical.

§ [version.syn]

Add a feature-test macro to `[version.syn]` as follows:



```
#define __cpp_lib_cmath 20XXXXL // also in <cmath>, <cstdlib>
```



DRAFTING NOTE: Although the proposal also adds a `__STDC_VERSION_MATH_H__` macro from C23, this cannot be reliably used to detect the new functions; it could have also been leaked through a `<math.h>` header from somewhere, without e.g. `std::roundeven` actually existing.

Bump the feature-test macro for mathematical special functions:



```
#define __cpp_lib_math_special_functions 201603L 20XXXXL // also in <cmath>
```



DRAFTING NOTE: This queries existence of new overloads such as `ellint_3f32`.

§ `[cmath.syn]`



```
#define __STDC_VERSION_MATH_H__ 202311L

#define HUGE_VAL see below
#define HUGE_VALF see below
#define HUGE_VALL see below
#define HUGE_VAL_FN see below // optional
#define INFINITY see below
#define NAN see below

#define FP_INFINITE see below
#define FP_NAN see below
#define FP_NORMAL see below
#define FP_SUBNORMAL see below
#define FP_ZERO see below

#define FP_INT_UPWARD see below
#define FP_INT_DOWNWARD see below
#define FP_INT_TOWARDZERO see below
#define FP_INT_TONEARESTFROMZERO see below
#define FP_INT_TONEAREST see below

#define FP_FAST_FMA see below
#define FP_FAST_FMAF see below
#define FP_FAST_FMAL see below
#define FP_FAST_FMAFN see below
#define FP_FAST_FADD see below
#define FP_FAST_FADDL see below
#define FP_FAST_DADDL see below
#define FP_FAST_FMAADDN see below
#define FP_FAST_FSUB see below
#define FP_FAST_FSUBL see below
#define FP_FAST_DSUBL see below
#define FP_FAST_FMSUBN see below
```



```
#define FP_FAST_FFMUL see below
#define FP_FAST_FFMULL see below
#define FP_FAST_DMULL see below
#define FP_FAST_FMMULN see below
#define FP_FAST_FDIV see below
#define FP_FAST_FDIVL see below
#define FP_FAST_DDIVL see below
#define FP_FAST_FMDIVN see below
#define FP_FAST_FSQRT see below
#define FP_FAST_FSQRTL see below
#define FP_FAST_DSQRTL see below
#define FP_FAST_FMSQRTN see below
#define FP_FAST_FFMA see below
#define FP_FAST_FFMAL see below
#define FP_FAST_DFMAL see below
#define FP_FAST_FMFMAN see below

#define FP_ILOGB0 see below
#define FP_ILOGBNAN see below

#define MATH_ERRNO see below
#define MATH_ERREXCEPT see below

#define math_errhandling see below

namespace std {
    using float_t = see below;
    using double_t = see below;
    using long_double_t = see below;

    constexpr floating-point-type acos(floating-point-type x);
    constexpr float acosf(float x);
    constexpr long double acosl(long double x);
    constexpr floatN_t acosfN(floatN_t x);

    constexpr floating-point-type asin(floating-point-type x);
    constexpr float asinf(float x);
    constexpr long double asinl(long double x);
    constexpr floatN_t asinfN(floatN_t x);

    constexpr floating-point-type atan(floating-point-type x);
    constexpr float atanf(float x);
    constexpr long double atanl(long double x);
    constexpr floatN_t atanfN(floatN_t x);

    constexpr floating-point-type atan2(floating-point-type y, floating-point-type x);
    constexpr float atan2f(float y, float x);
    constexpr long double atan2l(long double y, long double x);
    constexpr floatN_t atan2fN(floatN_t y, floatN_t x);

    constexpr floating-point-type cos(floating-point-type x);
    constexpr float cosf(float x);
    constexpr long double cosl(long double x);
    constexpr floatN_t cosfN(floatN_t x);
```



```
constexpr floating-point-type sin(floating-point-type x);
constexpr float sinf(float x);
constexpr long double sinl(long double x);
constexpr floatN_t sinfN(floatN_t x);

constexpr floating-point-type tan(floating-point-type x);
constexpr float tanf(float x);
constexpr long double tanl(long double x);
constexpr floatN_t tanfN(floatN_t x);

constexpr floating-point-type acospi(floating-point-type x);
constexpr float acospif(float x);
constexpr long double acospil(long double x);
constexpr floatN_t acospifN(floatN_t x);

constexpr floating-point-type asinpi(floating-point-type x);
constexpr float asinpif(float x);
constexpr long double asinpil(long double x);
constexpr floatN_t asinpifN(floatN_t x);

constexpr floating-point-type atanpi(floating-point-type x);
constexpr float atanpif(float x);
constexpr long double atanpil(long double x);
constexpr floatN_t atanfN(floatN_t x);

constexpr floating-point-type atan2pi(floating-point-type y, floating-po.
constexpr float atan2pif(float y, float x);
constexpr long double atan2pil(long double y, long double x);
constexpr floatN_t atan2pifN(floatN_t y, floatN_t x);

constexpr floating-point-type cospi(floating-point-type x);
constexpr float cospif(float x);
constexpr long double cospil(long double x);
constexpr floatN_t cospifN(floatN_t x);

constexpr floating-point-type sinpi(floating-point-type x);
constexpr float sinpif(float x);
constexpr long double sinpil(long double x);
constexpr floatN_t sinpifN(floatN_t x);

constexpr floating-point-type tanpi(floating-point-type x);
constexpr float tanpif(float x);
constexpr long double tanpil(long double x);
constexpr floatN_t tanpifN(floatN_t x);

constexpr floating-point-type acosh(floating-point-type x);
constexpr float acoshf(float x);
constexpr long double acoshl(long double x);
constexpr floatN_t acoshfN(floatN_t x);

constexpr floating-point-type asinh(floating-point-type x);
constexpr float asinhf(float x);
constexpr long double asinhl(long double x);
constexpr floatN_t asinhfN(floatN_t x);
```



```
constexpr floating-point-type atanh(floating-point-type x);
constexpr float               atanhf(float x);
constexpr long double         atanhL(long double x);
constexpr floatN_t            atanhfN(floatN_t x);

constexpr floating-point-type cosh(floating-point-type x);
constexpr float               coshf(float x);
constexpr long double         coshl(long double x);
constexpr floatN_t            coshfN(floatN_t x);

constexpr floating-point-type sinh(floating-point-type x);
constexpr float               sinhf(float x);
constexpr long double         sinhl(long double x);
constexpr floatN_t            sinhfN(floatN_t x);

constexpr floating-point-type tanh(floating-point-type x);
constexpr float               tanhf(float x);
constexpr long double         tanhl(long double x);
constexpr floatN_t            tanhfN(floatN_t x);

constexpr floating-point-type exp(floating-point-type x);
constexpr float               expf(float x);
constexpr long double         expl(long double x);
constexpr floatN_t            expfN(floatN_t x);

constexpr floating-point-type exp10(floating-point-type x);
constexpr float               exp10f(float x);
constexpr long double         exp10L(long double x);
constexpr floatN_t            exp10fN(floatN_t x);

constexpr floating-point-type exp10m1(floating-point-type x);
constexpr float               exp10m1f(float x);
constexpr long double         exp10m1L(long double x);
constexpr floatN_t            exp10m1fN(floatN_t x);

constexpr floating-point-type exp2(floating-point-type x);
constexpr float               exp2f(float x);
constexpr long double         exp2L(long double x);
constexpr floatN_t            exp2fN(floatN_t x);

constexpr floating-point-type exp2m1(floating-point-type x);
constexpr float               exp2m1f(float x);
constexpr long double         exp2m1L(long double x);
constexpr floatN_t            exp2m1fN(floatN_t x);

constexpr floating-point-type expm1(floating-point-type x);
constexpr float               expm1f(float x);
constexpr long double         expm1L(long double x);
constexpr floatN_t            expm1fN(floatN_t x);

constexpr floating-point-type frexp(floating-point-type value, int* exp);
constexpr float               frexpf(float value, int* exp);
constexpr long double         frexpl(long double value, int* exp);
constexpr floatN_t            frexpfN(floatN_t value, int* exp);
```



```
constexpr int ilogb(floating-point-type x);
constexpr int ilogbf(float x);
constexpr int ilogbl(long double x);
constexpr int ilogb(floatN_t x);

constexpr floating-point-type ldexp(floating-point-type x, int exp);
constexpr float ldexpf(float x, int exp);
constexpr long double ldexpl(long double x, int exp);
constexpr floatN_t ldexpfN(floatN_t x, int exp);

constexpr floating-point-type log(floating-point-type x);
constexpr float logf(float x);
constexpr long double logl(long double x);
constexpr floatN_t logfN(floatN_t x);

constexpr floating-point-type log10(floating-point-type x);
constexpr float log10f(float x);
constexpr long double log10l(long double x);
constexpr floatN_t log10fN(floatN_t x);

constexpr floating-point-type log10p1(floating-point-type x);
constexpr float log10p1f(float x);
constexpr long double log10p1l(long double x);
constexpr floatN_t log10p1fN(floatN_t x);

constexpr floating-point-type log1p(floating-point-type x);
constexpr float log1pf(float x);
constexpr long double log1pl(long double x);
constexpr floatN_t log1pfN(floatN_t x);

constexpr floating-point-type logp1(floating-point-type x);
constexpr float logp1f(float x);
constexpr long double logp1l(long double x);
constexpr floatN_t logp1fN(floatN_t x);

constexpr floating-point-type log2(floating-point-type x);
constexpr float log2f(float x);
constexpr long double log2l(long double x);
constexpr floatN_t log2fN(floatN_t x);

constexpr floating-point-type log2p1(floating-point-type x);
constexpr float log2p1f(float x);
constexpr long double log2p1l(long double x);
constexpr floatN_t log2p1fN(floatN_t x);

constexpr floating-point-type logb(floating-point-type x);
constexpr float logbf(float x);
constexpr long double logbl(long double x);
constexpr floatN_t logbfN(floatN_t x);

constexpr floating-point-type modf(floating-point-type value, floating-point-type iptr);
constexpr float modff(float value, float* iptr);
constexpr long double modfl(long double value, long double* iptr);
constexpr floatN_t modfN(floatN_t value, floatN_t* iptr);
```



```
constexpr floating-point-type scalbn(floating-point-type x, int n);
constexpr float scalbnf(float x, int n);
constexpr long double scalbnl(long double x, int n);
constexpr floatN_t scalbnfN(floatN_t x, int n);

constexpr floating-point-type scalbln(floating-point-type x, long int n);
constexpr float scalblnf(float x, long int n);
constexpr long double scalblnl(long double x, long int n);
constexpr floatN_t scalblnfN(floatN_t x);

constexpr floating-point-type cbrt(floating-point-type x);
constexpr float cbrtf(float x);
constexpr long double cbrtl(long double x);
constexpr floatN_t cbrtfN(floatN_t x);

constexpr floating-point-type compoundn(floating-point-type x, long long int n);
constexpr float compoundn(float x, long long int n);
constexpr long double compoundn(long double x, long long int n);
constexpr floatN_t compoundnfN(floatN_t x, long long int n);

// [c.math.abs], absolute values
constexpr int abs(int j); //
constexpr long int abs(long int j); //
constexpr long long int abs(long long int j); //
constexpr floating-point-type abs(floating-point-type j); //

constexpr floating-point-type fabs(floating-point-type x);
constexpr float fabsf(float x);
constexpr long double fabsl(long double x);
constexpr floatN_t fabsfN(floatN_t x);

constexpr floating-point-type hypot(floating-point-type x, floating-point-type y);
constexpr float hypotf(float x, float y);
constexpr long double hypotl(long double x, long double y);
constexpr floatN_t hypotfN(floatN_t x, floatN_t y);

// [c.math.hypot3], three-dimensional hypotenuse
constexpr floating-point-type hypot(floating-point-type x, floating-point-type y,
                                     floating-point-type z);

constexpr floating-point-type pow(floating-point-type x, floating-point-type y);
constexpr float powf(float x, float y);
constexpr long double powl(long double x, long double y);
constexpr floatN_t powfN(floatN_t x, floatN_t y);

constexpr floating-point-type pown(floating-point-type x, long long int n);
constexpr float pownf(float x, long long int n);
constexpr long double pownl(long double x, long long int n);
constexpr floatN_t pownfN(floatN_t x, long long int n);

constexpr floating-point-type powr(floating-point-type x, floating-point-type y);
constexpr float powrf(float x, float y);
constexpr long double powrl(long double x, long double y);
constexpr floatN_t powrfN(floatN_t x, floatN_t y);
```



```

constexpr floating-point-type rootn(floating-point-type x, long long int n);
constexpr float                rootnf(float x, long long int n);
constexpr long double          rootnl(long double x, long long int n);
constexpr floatN_t             rootnfN(floatN_t x, long long int n);

constexpr floating-point-type rsqrt(floating-point-type x);
constexpr float                rsqrtf(float x);
constexpr long double          rsqrtl(long double x);
constexpr floatN_t             rsqrtfN(floatN_t x);

constexpr floating-point-type sqrt(floating-point-type x);
constexpr float                sqrtf(float x);
constexpr long double          sqrtl(long double x);
constexpr floatN_t             sqrtfN(floatN_t x);

constexpr floating-point-type erf(floating-point-type x);
constexpr float                erff(float x);
constexpr long double          erfl(long double x);
constexpr floatN_t             erffN(floatN_t x);

constexpr floating-point-type erfc(floating-point-type x);
constexpr float                erfcf(float x);
constexpr long double          erfcl(long double x);
constexpr floatN_t             erfcfN(floatN_t x);

constexpr floating-point-type lgamma(floating-point-type x);
constexpr float                lgammaf(float x);
constexpr long double          lgammal(long double x);
constexpr floatN_t             lgammafN(floatN_t x);

constexpr floating-point-type tgamma(floating-point-type x);
constexpr float                tgammaf(float x);
constexpr long double          tgammal(long double x);
constexpr floatN_t             tgammafN(floatN_t x);

constexpr floating-point-type ceil(floating-point-type x);
constexpr float                ceilf(float x);
constexpr long double          ceill(long double x);
constexpr floatN_t             ceilfN(floatN_t x);

constexpr floating-point-type floor(floating-point-type x);
constexpr float                floorf(float x);
constexpr long double          floorl(long double x);
constexpr floatN_t             floorfN(floatN_t x);

floating-point-type nearbyint(floating-point-type x);
float                nearbyintf(float x);
long double          nearbyintl(long double x);
floatN_t             nearbyintfN(floatN_t x);

floating-point-type rint(floating-point-type x);
float                rintf(float x);
long double          rintl(long double x);
floatN_t             rintfN(floatN_t x);

```




```
long int lrint(floating-point-type x);
long int lrintf(float x);
long int lrintl(long double x);
long int lrintfN(floatN_t x);

long long int llrint(floating-point-type x);
long long int llrintf(float x);
long long int llrintl(long double x);
long long int llrintfN(floatN_t x);

constexpr floating-point-type round(floating-point-type x);
constexpr float roundf(float x);
constexpr long double roundl(long double x);
constexpr floatN_t roundfN(floatN_t x);

constexpr long int lround(floating-point-type x);
constexpr long int lroundf(float x);
constexpr long int lroundl(long double x);
constexpr long int lroundfN(floatN_t x);

constexpr long long int llround(floating-point-type x);
constexpr long long int llroundf(float x);
constexpr long long int llroundl(long double x);
constexpr long long int llroundfN(floatN_t x);

constexpr floating-point-type roundeven(floating-point-type x);
constexpr float roundevenf(float x);
constexpr long double roundevenl(long double x);
constexpr floatN_t roundevenfN(floatN_t x);

constexpr floating-point-type trunc(floating-point-type x);
constexpr float truncf(float x);
constexpr long double truncfN(floatN_t x);

constexpr floating-point-type fromfp(floating-point-type x, int rnd, unsigned int width);
constexpr float fromfpf(float x, int rnd, unsigned int width);
constexpr long double fromfpl(long double x, int rnd, unsigned int width);
constexpr floatN_t fromfpfN(floatN_t x, int rnd, unsigned int width);

constexpr floating-point-type ufromfp(floating-point-type x, int rnd, unsigned int width);
constexpr float ufromfpf(float x, int rnd, unsigned int width);
constexpr long double ufromfpl(long double x, int rnd, unsigned int width);
constexpr floatN_t ufromfpfN(floatN_t x, int rnd, unsigned int width);

constexpr floating-point-type fromfpx(floating-point-type x, int rnd, unsigned int width);
constexpr float fromfpxf(float x, int rnd, unsigned int width);
constexpr long double fromfpxl(long double x, int rnd, unsigned int width);
constexpr floatN_t fromfpxfN(floatN_t x, int rnd, unsigned int width);

constexpr floating-point-type ufromfpx(floating-point-type x, int rnd, unsigned int width);
constexpr float ufromfpxf(float x, int rnd, unsigned int width);
constexpr long double ufromfpxl(long double x, int rnd, unsigned int width);
constexpr floatN_t ufromfpxfN(floatN_t x, int rnd, unsigned int width);
```




```
constexpr floating-point-type fmod(floating-point-type x, floating-point-type y);
constexpr float fmodf(float x, float y);
constexpr long double fmodl(long double x, long double y);
constexpr floatN_t fmodfN(floatN_t x, floatN_t y);

constexpr floating-point-type remainder(floating-point-type x, floating-point-type y);
constexpr float remainderf(float x, float y);
constexpr long double remainderl(long double x, long double y);
constexpr floatN_t remainderfN(floatN_t x, floatN_t y);

constexpr floating-point-type remquo(floating-point-type x, floating-point-type y, int* quo);
constexpr float remquof(float x, float y, int* quo);
constexpr long double remquol(long double x, long double y, int* quo);
constexpr floatN_t remquofN(floatN_t x, floatN_t y, int* quo);

constexpr floating-point-type copysign(floating-point-type x, floating-point-type y);
constexpr float copysignf(float x, float y);
constexpr long double copysignl(long double x, long double y);
constexpr floatN_t copysignfN(floatN_t x, floatN_t y);

double nan(const char* tagp);
float nanf(const char* tagp);
long double nanl(const char* tagp);
floatN_t nanfN(const char* tagp);

constexpr floating-point-type nextafter(floating-point-type x, floating-point-type y);
constexpr float nextafterf(float x, float y);
constexpr long double nextafterl(long double x, long double y);
constexpr floatN_t nextafterfN(floatN_t x, floatN_t y);

constexpr floating-point-type nexttoward(floating-point-type x, long double y);
constexpr float nexttowardf(float x, long double y);
constexpr long double nexttowardl(long double x, long double y);
constexpr floatN_t nexttowardfN(floatN_t x, floatN_t y);

constexpr floating-point-type nextup(floating-point-type x);
constexpr float nextupf(float x);
constexpr long double nextupl(long double x);
constexpr floatN_t nextupfN(floatN_t x);

constexpr floating-point-type nextdown(floating-point-type x);
constexpr float nextdownf(float x);
constexpr long double nextdownl(long double x);
constexpr floatN_t nextdownfN(floatN_t x);

constexpr int canonicalize(floating-point-type* cx, const floating-point-type* x);
constexpr int canonicalizef(float cx, const float* x);
constexpr int canonicalizel(long double cx, const long double* x);
constexpr int canonicalizefN(floatN_t* cx, const floatN_t* x);

constexpr floating-point-type fdim(floating-point-type x, floating-point-type y);
constexpr float fdimf(float x, float y);
constexpr long double fdiml(long double x, long double y);
constexpr floatN_t fdimfN(floatN_t x, floatN_t y);
```



```
constexpr floating-point-type fmax(floating-point-type x, floating-point-type y);
constexpr float fmaxf(float x, float y);
constexpr long double fmaxl(long double x, long double y);
constexpr floatN_t fmaxfN(floatN_t x, floatN_t y);

constexpr floating-point-type fmin(floating-point-type x, floating-point-type y);
constexpr float fminf(float x, float y);
constexpr long double fminl(long double x, long double y);
constexpr floatN_t fminfN(floatN_t x, floatN_t y);

constexpr floating-point-type fmaximum(floating-point-type x, floating-point-type y);
constexpr float fmaximumf(float x, float y);
constexpr long double fmaximuml(long double x, long double y);
constexpr floatN_t fmaximumfN(floatN_t x, floatN_t y);

constexpr floating-point-type fmaximum_num(floating-point-type x, floating-point-type y);
constexpr float fmaximum_numf(float x, float y);
constexpr long double fmaximum_numl(long double x, long double y);
constexpr floatN_t fmaximum_numfN(floatN_t x, floatN_t y);

constexpr floating-point-type fmaximum_mag(floating-point-type x, floating-point-type y);
constexpr float fmaximum_magf(float x, float y);
constexpr long double fmaximum_magl(long double x, long double y);
constexpr floatN_t fmaximum_magfN(floatN_t x, floatN_t y);

constexpr floating-point-type fmaximum_mag_num(floating-point-type x, floating-point-type y);
constexpr float fmaximum_mag_numf(float x, float y);
constexpr long double fmaximum_mag_numl(long double x, long double y);
constexpr floatN_t fmaximum_mag_numfN(floatN_t x, floatN_t y);

constexpr floating-point-type fminimum(floating-point-type x, floating-point-type y);
constexpr float fminimumf(float x, float y);
constexpr long double fminimuml(long double x, long double y);
constexpr floatN_t fminimumfN(floatN_t x, floatN_t y);

constexpr floating-point-type fminimum_num(floating-point-type x, floating-point-type y);
constexpr float fminimum_numf(float x, float y);
constexpr long double fminimum_numl(long double x, long double y);
constexpr floatN_t fminimum_numfN(floatN_t x, floatN_t y);

constexpr floating-point-type fminimum_mag(floating-point-type x, floating-point-type y);
constexpr float fminimum_magf(float x, float y);
constexpr long double fminimum_magl(long double x, long double y);
constexpr floatN_t fminimum_magfN(floatN_t x, floatN_t y);

constexpr floating-point-type fminimum_mag_num(floating-point-type x, floating-point-type y);
constexpr float fminimum_mag_numf(float x, float y);
constexpr long double fminimum_mag_numl(long double x, long double y);
constexpr floatN_t fminimum_mag_numfN(floatN_t x, floatN_t y);

constexpr floating-point-type fma(floating-point-type x, floating-point-type y, floating-point-type z);
constexpr float fmaf(float x, float y, float z);
constexpr long double fmal(long double x, long double y, long double z);
```



```
constexpr floating-point-type lerp(floating-point-type a, floating-point-type t, floating-point-type b) noexcept;
```



```
// [c.math.fpclass], classification / comparison functions
constexpr int fpclassify(floating-point-type x);
constexpr bool iscanonical(floating-point-type x);
constexpr bool isfinite(floating-point-type x);
constexpr bool isinf(floating-point-type x);
constexpr bool isnan(floating-point-type x);
constexpr bool isnormal(floating-point-type x);
constexpr bool signbit(floating-point-type x);
constexpr bool issignaling(floating-point-type x);
constexpr bool issubnormal(floating-point-type x);
constexpr bool iszero(floating-point-type x);
constexpr bool isgreater(floating-point-type x, floating-point-type y);
constexpr bool isgreaterequal(floating-point-type x, floating-point-type y);
constexpr bool isless(floating-point-type x, floating-point-type y);
constexpr bool islessequal(floating-point-type x, floating-point-type y);
constexpr bool islessgreater(floating-point-type x, floating-point-type y);
constexpr bool isunordered(floating-point-type x, floating-point-type y);
constexpr bool iseqsig(floating-point-type x, floating-point-type y);

// ISO/IEC 60559 specific functions
constexpr bool totalorder(const floating-point-type* x,
                          const floating-point-type* y);
constexpr bool totalorderf(const float* x, const float* y);
constexpr bool totalorderl(const long double* x, const long double* y);
constexpr bool totalorderfN(const floatN_t* x, floatN_t* y);

constexpr bool totalordermag(const floating-point-type* x,
                             const floating-point-type* y);
constexpr bool totalordermagf(const float* x, const float* y);
constexpr bool totalordermagl(const long double* x, const long double* y);
constexpr bool totalordermagfN(const floatN_t* x, floatN_t* y);

constexpr floating-point-type getpayload(const floating-point-type* x);
constexpr float getpayloadf(const float* x);
constexpr long double getpayloadl(const long double* x);
constexpr floatN_t getpayloadfN(const floatN_t* x);

constexpr int setpayload(floating-point-type* res, floating-point-type pl);
constexpr int setpayloadf(float* res, float pl);
constexpr int setpayloadl(long double* res, long double pl);
constexpr int setpayloadfN(floatN_t* res, floatN_t pl);

// [sf.cmath], mathematical special functions

// [sf.cmath.assoc.laguerre], associated Laguerre polynomials
floating-point-type assoc_laguerre(unsigned n, unsigned m, floating-point-type x);
float assoc_laguerref(unsigned n, unsigned m, float x);
long double assoc_laguerrel(unsigned n, unsigned m, long double x);
floatN_t assoc_laguerrefN(unsigned n, unsigned m, floatN_t x);

// [sf.cmath.assoc.legendre], associated Legendre functions
floating-point-type assoc_legendre(unsigned l, unsigned m, floating-point-type x);
float assoc_legendref(unsigned l, unsigned m, float x);
long double assoc_legendrel(unsigned l, unsigned m, long double x);
```



```

floatN_t      assoc_legendrefN(unsigned l, unsigned m, floatN_t x);

// [sf.cmath.beta], beta function
floating-point-type beta(floating-point-type x, floating-point-type y);
float          betaf(float x, float y);
long double    betal(long double x, long double y);
floatN_t       betafN(floatN_t x, floatN_t y);

// [sf.cmath.comp.ellint.1], complete elliptic integral of the first kind
floating-point-type comp_ellint_1(floating-point-type k);
float          comp_ellint_1f(float k);
long double    comp_ellint_1l(long double k);
floatN_t       comp_ellint_1fN(floatN_t k);

// [sf.cmath.comp.ellint.2], complete elliptic integral of the second kind
floating-point-type comp_ellint_2(floating-point-type k);
float          comp_ellint_2f(float k);
long double    comp_ellint_2l(long double k);
floatN_t       comp_ellint_2fN(floatN_t k);

// [sf.cmath.comp.ellint.3], complete elliptic integral of the third kind
floating-point-type comp_ellint_3(floating-point-type k, floating-point-type nu);
float          comp_ellint_3f(float k, float nu);
long double    comp_ellint_3l(long double k, long double nu);
floatN_t       comp_ellint_3fN(floatN_t k, floatN_t nu);

// [sf.cmath.cyl.bessel.i], regular modified cylindrical Bessel functions
floating-point-type cyl_bessel_i(floating-point-type nu, floating-point-type x);
float          cyl_bessel_if(float nu, float x);
long double    cyl_bessel_il(long double nu, long double x);
floatN_t       cyl_bessel_ifN(floatN_t nu, floatN_t x);

// [sf.cmath.cyl.bessel.j], cylindrical Bessel functions of the first kind
floating-point-type cyl_bessel_j(floating-point-type nu, floating-point-type x);
float          cyl_bessel_jf(float nu, float x);
long double    cyl_bessel_jl(long double nu, long double x);
floatN_t       cyl_bessel_jfN(floatN_t nu, floatN_t x);

// [sf.cmath.cyl.bessel.k], irregular modified cylindrical Bessel functions
floating-point-type cyl_bessel_k(floating-point-type nu, floating-point-type x);
float          cyl_bessel_kf(float nu, float x);
long double    cyl_bessel_kl(long double nu, long double x);
floatN_t       cyl_bessel_kfN(floatN_t nu, floatN_t x);

// [sf.cmath.cyl.neumann], cylindrical Neumann functions
// cylindrical Bessel functions of the second kind
floating-point-type cyl_neumann(floating-point-type nu, floating-point-type x);
float          cyl_neumannf(float nu, float x);
long double    cyl_neumannl(long double nu, long double x);
floatN_t       cyl_neumannfN(floatN_t nu, floatN_t x);

// [sf.cmath.ellint.1], incomplete elliptic integral of the first kind
floating-point-type ellint_1(floating-point-type k, floating-point-type phi);
float          ellint_1f(float k, float phi);

```




```
long double      ellint_1l(long double k, long double phi);
floatN_t         ellint_1fN(floatN_t k, floatN_t phi);

// [sf.cmath.ellint.2], incomplete elliptic integral of the second kind
floating-point-type ellint_2(floating-point-type k, floating-point-type phi);
float              ellint_2f(float k, float phi);
long double        ellint_2l(long double k, long double phi);
floatN_t           ellint_2fN(floatN_t k, floatN_t phi);

// [sf.cmath.ellint.3], incomplete elliptic integral of the third kind
floating-point-type ellint_3(floating-point-type k, floating-point-type phi);
float              ellint_3f(float k, float nu, float phi);
long double        ellint_3l(long double k, long double nu, long double phi);
floatN_t           ellint_3fN(floatN_t k, floatN_t nu, floatN_t phi);

// [sf.cmath.expint], exponential integral
floating-point-type expint(floating-point-type x);
float              expintf(float x);
long double        expintl(long double x);
floatN_t           expintfN(floatN_t x);

// [sf.cmath.hermite], Hermite polynomials
floating-point-type hermite(unsigned n, floating-point-type x);
float              hermitef(unsigned n, float x);
long double        hermitel(unsigned n, long double x);
floatN_t           hermitefN(unsigned n, floatN_t x);

// [sf.cmath.laguerre], Laguerre polynomials
floating-point-type laguerre(unsigned n, floating-point-type x);
float              laguerref(unsigned n, float x);
long double        laguerrel(unsigned n, long double x);
floatN_t           laguerrefN(unsigned n, floatN_t x);

// [sf.cmath.legendre], Legendre polynomials
floating-point-type legendre(unsigned l, floating-point-type x);
float              legendref(unsigned l, float x);
long double        legendrel(unsigned l, long double x);
floatN_t           legendrefN(unsigned l, floatN_t x);

// [sf.cmath.riemann.zeta], Riemann zeta function
floating-point-type riemann_zeta(floating-point-type x);
float              riemann_zetaf(float x);
long double        riemann_zetal(long double x);
floatN_t           riemann_zetafN(floatN_t x);

// [sf.cmath.sph.bessel], spherical Bessel functions of the first kind
floating-point-type sph_bessel(unsigned n, floating-point-type x);
float              sph_besself(unsigned n, float x);
long double        sph_bessell(unsigned n, long double x);
floatN_t           sph_besselfN(unsigned n, floatN_t x);

// [sf.cmath.sph.legendre], spherical associated Legendre functions
floating-point-type sph_legendre(unsigned l, unsigned m, floating-point-type x);
```

```

float          sph_legendref(unsigned l, unsigned m, float theta);
long double    sph_legendrel(unsigned l, unsigned m, long double theta);
floatN_t       sph_legendrefN(unsigned l, unsigned m, floatN_t theta);

// [sf.cmath.sph.neumann], spherical Neumann functions;
// spherical Bessel functions of the second kind
floating-point-type sph_neumann(unsigned n, floating-point-type x);
float              sph_neumannf(unsigned n, float x);
long double        sph_neumannl(unsigned n, long double x);
floatN_t           sph_neumannfN(unsigned n, floatN_t x);
}

```

Change [\[cmath.syn\]](#) paragraph 1 as follows:



The contents and meaning of the header `<cmath>` are a subset of the C standard library header `<math.h>` and only the declarations shown in the synopsis above are present, with the addition of a three-dimensional hypotenuse function, a linear interpolation function, and the mathematical special functions described in [\[sf.cmath\]](#).

Change [\[cmath.syn\]](#) paragraph 2 as follows:



~~For each function with at least one parameter of type *floating-point-type*, the implementation provides an overload for each cv-unqualified floating-point type ([basic.fundamental]).~~ Multiple functions above are specified using a type placeholder, for which the implementation provides an overload for each type in a set, where all uses of *floating-point-type* the type placeholder in the function signature are replaced with that floating-point type.

- For the type placeholder *floating-point-type*, the set of types is all cv-unqualified floating-point types.
- For the type placeholder *double-type*, the set of types is `double` and `long double`.
- For the type placeholder *floatN_t*, the set of types is `float16_t`, `float32_t`, `float64_t`, and `float128_t`, if defined. The *N* placeholder in the function name is replaced with the corresponding width in the type name.

Insert a new item immediately following [\[cmath.syn\]](#) paragraph 2:



¶ Let *common-floating-point-type* and *common-double-type* be the following exposition-only alias templates:

```

template<class... Ts>
    using common-floating-point-type = see below;
template<class... Ts>
    using common-double-type = see below;

```

¶ *Constraints:* Each type in the pack *Ts* is a cv-unqualified arithmetic type ([basic.fundamental]).

¶ Let R be defined as follows:

- For *common-floating-point-type*, let R be the floating-point type with the greatest floating-point conversion rank and greatest floating-point conversion subrank among the types in Ts, where integer types are considered to have the same floating-point conversion rank as `double`.
- For *common-double-type*, let R be `long double` if any of the types in Ts is `long double`, and `double` otherwise.

¶ Mandates: R is defined.

¶ Result: R.



DRAFTING NOTE: *common-floating-point-type* is just following the existing semantics of [\[cmath.syn\] paragraph 3](#) in the form of an exposition-only alias template.

common-double-type is meant to emulate the behavior of the `<tgmath.h>` macros `fadd`, `fsub`, etc.

For `dadd`, `dsub`, etc. no such thing is necessary because without decimal floating-point types, the behavior is always to convert all arguments to `long double`.

Change [\[cmath.syn\] paragraph 3](#) as follows:



For each function with at least one parameter ~~of type *floating-point-type* declared with a type placeholder~~, other than `abs`, the implementation also provides additional overloads sufficient to ensure that, if every argument corresponding to a *floating-point-type* type placeholder parameter has arithmetic type, then every such argument is effectively cast to ~~the floating-point type with the greatest floating-point conversion rank and greatest floating-point conversion subrank among the types of all such arguments, where arguments of integer type are considered to have the same floating-point conversion rank as `double`~~

- *common-double-type*`<Args...>` if the type placeholder is *double-type*, and
- *common-floating-point-type*`<Args...>` otherwise.

If ~~no such floating-point type with the greatest rank and subrank exists~~ that specialization of the alias template is ill-formed, then overload resolution does not result in a usable candidate ([\[over.match.general\]](#)) from the overloads provided by the implementation.

Add a new paragraphs immediately following [\[cmath.syn\] paragraph 3](#):



The implementation may provide additional overloads of *padd*, *psub*, *pmul*, *pdiv*, *pfma*, and *psqrt*, for any prefix *p*, with parameters of cv-unqualified arithmetic types to ensure consistent behavior with the corresponding macro in C header `<tgmath.h>`.



DRAFTING NOTE: While the implementation is generally free to provide additional overloads, this is not allowed to alter the behavior of a conforming program. If an implementation provided an extended floating-point type `_Decimal128`, `dadd` would need to convert arguments of type `_Decimal128` to `long double`, but the C23 behavior is to call `dadd128`.

This would not be the case if `_Decimal128` was a compiler extension, in which case the program isn't standard C++ anyway, and the implementation can do whatever it wants.



The functions whose names begin with `totalorder`, `totalordermag`, `getpayload`, or `setpayload` are only provided if `numeric_limits<T>::is_iec559` is `true` for the floating-point type `T` in the function signature (possibly following the placeholder substitution described above).

§ [c.math.narrow]

Add a new subclause as follows:



Narrowing operations

[c.math.narrow]

```
constexpr float    fop(long double x /* ... */);
constexpr double   dop(long double x /* ... */);
constexpr floatN_t fNop(see below x /* ... */);
constexpr floatN_t fNopfM(floatM_t x /* ... */);
```

¶ Let *op* be a placeholder for add, sub, mul, div, fma, or sqrt.

¶ For each of the functions *fNop* and *fNopfM*, the implementation provides an overload for every pair *N* and *M* if *N* < *M* and the aliases `floatN_t` and `floatM_t` are defined, where every placeholder *N* and *M* in the function signature is replaced with the corresponding integers.

¶ For each function *fMopfN*, the implementation also defines a macro `FP_FAST_FM_upFN`, where *up* is *op* in uppercase.

¶ *Effects:* Each of the macros and overloads *fop* and *dop* has the same behavior as the function *fopl* and *dopl*, respectively. Each of the overloads *fNop* has the same behavior as the corresponding function *fNopfM* with the same parameter types.

¶ [Example: If only `float32_t`, `float64_t`, and `float128_t` are defined, the implementation provides the overloads

```
#define FP_FAST_F32SQRTF64 see above
#define FP_FAST_F32SQRTF128 see above
#define FP_FAST_F64SQRTF128 see above

constexpr float32_t f32sqrt(float64_t x);
constexpr float32_t f32sqrt(float128_t x);
constexpr float64_t f64sqrt(float128_t x);
```



```
constexpr float32_t f32sqrtf64(float64_t x);
constexpr float32_t f32sqrtf128(float128_t x);
constexpr float64_t f64sqrtf128(float128_t x);
```

where *op* is sqrt. — *end example*

```
template<class T, class F1, class F2>
    constexpr T fadd(F1 x, F2 y);
template<class T, class F1, class F2>
    constexpr T fsub(F1 x, F2 y);
template<class T, class F1, class F2>
    constexpr T fmul(F1 x, F2 y);
template<class T, class F1>
    constexpr T fdiv(F1 x, F2 y);
template<class T, class F1, class F2, class F3>
    constexpr T fma(F1 x, F2 y, F3 z);
template<class T, class F1>
    constexpr T fsqrt(F1 x);
```

¶ Let *Fs* be a pack of template type parameters following *T*. Let *F* be the type determined as follows:

- If *T* is `double` or if any type in *Fs* is `long double`, *F* is `long double`.
- Otherwise, if *T* is `float`, *F* is `double`.
- Otherwise, if *T* is an extended floating-point type named by an alias `std::floatN_t` ([`stdfloat.syn`]), *F* is the type named by `std::floatM_t`, where $M = N \times 2$, if such an alias is defined.
- Otherwise, *F* is implementation-defined and may not exist.

¶ *Constraints:*

- Each type in *Fs* is a cv-unqualified arithmetic type.
- *F* exists.
- `((numeric_limits<Fs>::radix == numeric_limits<F>::radix) && ...)` is `true`.
- *T* is a cv-unqualified floating-point type other than `long double`.

[*Example:* It is implementation-defined whether the constraints of `fadd<bfloat16_t, float32_t>` are satisfied. — *end example*]

¶ *Effects:* All parameters are converted to prvalues of type *F*. Each of the function templates has the same behavior as the corresponding non-template overload with the same name, if that overload had return type *T* and parameters all of type *F*.

¶ *Remarks:* The `FP_FAST` macros also relate to specializations of these function templates.

[*Example:* `FP_FAST_DSQRTL` relates to `dsqrtl` and `fsqrt<double, long double>`. — *end example*]

§ [sf.cmath]



For every overload set in this subclause, add the same overload as shown in the synopsis changes above:



EXAMPLE: Change the declaration in [sf.cmath.assoc.laguerre] as follows:



```
floating-point-type assoc_laguerre(unsigned n, unsigned m, floating-  
float               assoc_laguerref(unsigned n, unsigned m, float x)  
long double         assoc_laguerrel(unsigned n, unsigned m, long dou  
floatN_t             assoc_laguerrefN(unsigned n, unsigned m, floatN_
```

§ 6. Acknowledgements

Thanks to Hubert Tong for reviewing the library wording and pointing out asymmetries with the behavior of `<tgmath.h>` macros.

§ 7. References

[N5014] *Thomas Köppe*. Working Draft, Programming Languages — C++ (2025-08-05)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5014.pdf>

[P3348R4] *Jonathan Wakely*. C++26 should refer to C23 not C17 (2025-06-19)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3348r4.pdf>

[P3008R6] *Gonzalo Brito Gadeschi, David Sankel*. Atomic floating-point min/max (2025-06-19)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3008r6.html>